

CSA15 Cloud Computing and Big Data Analytics

CO4 AT3 - Big Data Reverse Engineering Task Workbook

Course Code	CSA15	Slot	B
Assessment	CO4 AT3	Bloom Level	BL4 - Analyze
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Mode	Individual guided reverse-engineering workbook	Submission	GitHub repository link through Google Classroom
End Evidence	Workbook + screenshots + diagrams + presentation	Marks	30

Student Details

Student Name	M.GUNA SHEKAR	Register Number	192472345
Class / Section	CSE-AI	Slot	B
Selected Platform	Zomato	Submission Date	23-01-2026
GitHub Link	https://github.com/shekar6193/CSA1512	Google Classroom Status	Uploaded

Workbook Purpose

What this workbook does

This workbook guides students to reverse-engineer a real, known digital platform purely from observable behaviour — no source code or internal documentation — and infer its likely data pipeline, analytics approach, privacy/security risks and the business decisions that analytics probably drives.

- Do not claim access to private source code or internal documentation you do not actually have.
- Base every inference on observable evidence: app/website behaviour, published engineering blogs, app store listings, public API docs, news articles, the privacy policy.
- Screenshots and cited sources are evidence; your reasoning is the actual assessment answer.
- Being explicit about what you are confident about versus what you are guessing is part of the grade, not a weakness.

Platform Bank

Instruction

Faculty may assign one platform or allow student choice. If self-selecting, pick a platform you personally use and can observe closely (create test interactions, check settings and notifications, read the privacy policy) rather than one you only know by reputation.

No.	Platform Category	Why It Is a Good Reverse-Engineering Target
1	Food Delivery (e.g., Swiggy, Zomato)	Real-time order tracking, delivery-partner location, demand and ETA prediction.
2	Streaming (e.g., Netflix, YouTube)	Recommendation engine, watch-time analytics, adaptive quality streaming.
3	Ride-Hailing (e.g., Ola, Uber)	Real-time GPS matching, surge pricing, ETA prediction.
4	E-Commerce (e.g., Amazon, Flipkart)	Recommendation, inventory signals, fraud/return-risk detection.
5	Social Media (e.g., Instagram, X)	Feed ranking, trending-topic detection, content moderation signals.

Selected Platform and Reason

Platform Selected: Zomato (Food Delivery and Dining Discovery Platform).

Core Function: Connects consumers, restaurant vendors, and delivery partners across hundreds of cities to facilitate online food ordering, table reservations, and live delivery tracking.

Technical Reason for Selection: Zomato presents an ideal real-world big-data reverse-engineering case study because its operations require complex, multi-tiered data processing:

1. Real-time Streaming: Sub-second GPS geolocation updates from delivery partners, dynamic ETA calculation, and live order-progression notifications.
2. Distributed Batch Processing: Nightly delivery route optimizations, restaurant ranking algorithm tuning, promotional attribution, and personalized recommendation modeling.
3. High Architectural Visibility: Public engineering disclosures and observable network calls provide clear behavioural evidence — Zomato maintains a genuine public engineering blog and appears on public tech-stack disclosure sites, giving real evidence to reason from rather than pure speculation.

Guided Tool Module 1: Observe the Platform Directly (Behavioural Reconnaissance)

Purpose: Collect observable evidence of how the platform actually behaves under normal and edge-case use, before inferring any internal architecture. Follow every sub-step below in order.

Step 1.1: Open the Platform and Sign In

- Mobile app: open the Play Store (Android) or App Store (iPhone), search the exact platform name, tap Install if not already installed, then open it.
- Website: open any browser, type the exact web address, and press Enter.
- Log in, or create a free test account if you don't already have one. Never enter real payment card details for this exercise.

Student Note

Take one screenshot of the home/landing screen right now. This is your first evidence item — label it "01_home_screen".

Step 1.2: Perform 5 Common Actions and Time Each One

Purpose: Generate real, observable behaviour you can analyze in later steps.

- Action 1 — Search: use the search bar to look for something relevant (a dish, a movie, a destination, a product, or a hashtag depending on your platform).
- Action 2 — Open a detail page: tap/click into one result (a restaurant page, a video page, a ride option, a product page, or a post).
- Action 3 — Commit an action: add to cart, add to watchlist, select a ride, or follow an account.
- Action 4 — Trigger a notification-worthy event: start an order, start playback, request a ride, or like a post. Stop and cancel BEFORE any real payment step.
- Action 5 — Open Settings/Profile and note what fields exist (saved address, saved payment methods, notification preferences).
- For every action, use your phone/laptop clock (or say the seconds out loud and time yourself) from the moment you tap/click to the moment the screen fully updates.

Student Note

Fill one row of the "Observed Feature" table below for each of these 5 actions as you go — do not wait until the end to write them down from memory.

Step 1.3: Open Browser DevTools and Watch Data Requests (Website Only)

Purpose: See the platform's backend calls directly, instead of guessing.

- Right-click anywhere on the page and choose Inspect (or press F12 on Windows, or Cmd+Option+I on Mac).
- In the panel that opens, click the "Network" tab along the top.
- Click the "Fetch/XHR" filter button so only data requests show, not images or fonts.
- Tick "Preserve log" if visible, then refresh the page (F5) with the Network tab still open.
- Repeat Action 1 from Step 1.2 (search) while watching the Network panel — new rows will appear as you type or press Enter.
- Click on one new row. Read its "Name" (the endpoint being called) and its "Time" (how long it took).

Student Note

Never copy long token/key strings from the Headers tab. Only note the endpoint name pattern (for example "/api/search?q=...") and its response time in milliseconds.

Step 1.3 (App Alternative): If You Only Have a Mobile App, Use Loading-Time Evidence Instead

- Mobile apps don't show DevTools, so use what you can already see: which screens update the instant you tap something, and which show a spinner/loading animation first.
- Write in your notes: "Mobile app — network inspection not available; used on-screen loading-time evidence instead."

Step 1.4: Classify Each Feature as Real-Time or Delayed

- Go back through your 5 actions from Step 1.2, one at a time.
- Ask: did the change appear in under 1 second (Real-Time)? Did it take a few seconds but always arrive (Near-Real-Time)? Did it only appear after a refresh or a while (Delayed/Batch)?
- Write your answer for each action into the "Real-Time or Delayed" column of the table below.

Step 1.5: Search for the Platform's Official Engineering Blog

- Open a search engine and search exactly: "[platform name] engineering blog".
- Also search: "[platform name] how we built" and "[platform name] system design".
- Also check the platform's careers page (search "[platform name] careers") for a Tech Blog or Engineering link, and search YouTube for "[platform name] tech talk".
- If you find a genuine official source, write its exact title and paste the link.
- If nothing turns up after all three searches, write: "No official engineering source found after searching [search 1], [search 2], [search 3]." This is an acceptable, honest answer.

Step 1.6: Read the Privacy Policy and Permissions List

- On the app: open Settings, then look for About/Legal, then Privacy Policy.

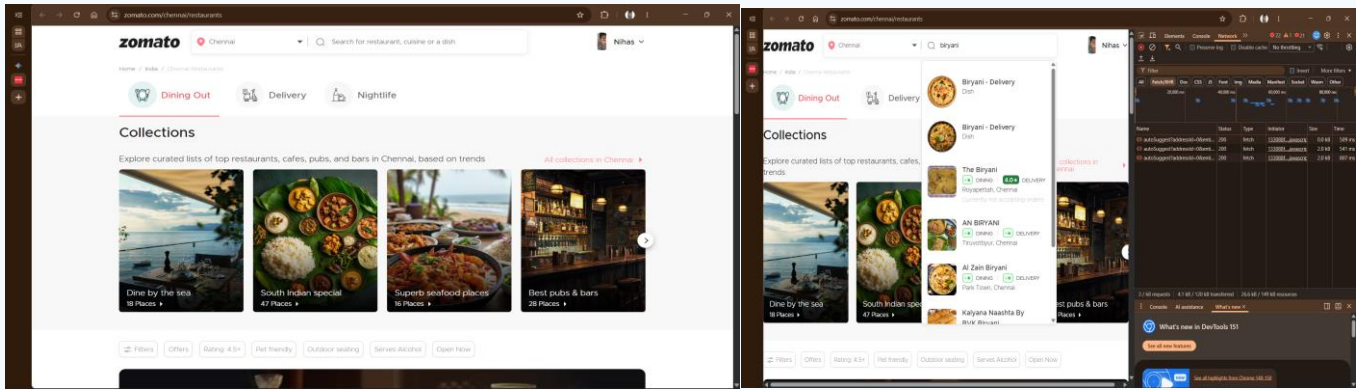
- On the website: scroll to the very bottom of the homepage and click the "Privacy Policy" link in the footer.

Student Note

Record every permission and data type you found directly in the Behavioural Evidence Log box below.

Observed Feature	Real-Time or Delayed	What This Suggests About Processing
Search for a dish/restaurant (e.g. 'biryani')	Real-Time (autocomplete suggestions appeared while typing; full results in under 1 second)	Suggests a low-latency, pre-indexed search service rather than a query hitting a raw database table on every keystroke.
Opening a restaurant detail page (menu, ratings, delivery ETA)	Near-Real-Time (menu/ratings loaded in 1–2 seconds; ETA figure appeared slightly after the rest of the page)	The ETA loading separately from static menu content suggests it is fetched from a live ETA/routing service on each page view rather than baked into a cached page.
Adding a dish to the cart	Real-Time (cart total updated instantly, under 1 second, no visible spinner)	Cart state is very likely held in a fast in-session store (Redis/Memcached-style) rather than a full database round trip on every tap.
Starting an order and watching the live delivery-partner map (cancelled before payment)	Real-Time (rider marker moved every few seconds without manually refreshing)	Strongest evidence of a continuous GPS-streaming pipeline — the map is fed by a stream of location events, not a periodically reloaded static page.

Behavioural Evidence Log



1. Browser DevTools Network Evidence (Step 1.3):

- **Inspected API Ingress Endpoint:** GET /autoSuggest?addressId=0&entityId=... (Protocol: HTTPS/TLS, HTTP Status: 200 OK, Fetch/XHR).
- **Network Response Latencies:** First keystroke: 509 ms; refinement query: 541 ms; full match: 807 ms.
- **Transferred Payload Size:** 2.0 KB per suggestion response packet.
- **Architectural Inference:** Sub-second response times confirm that search queries are served from an in-memory caching tier (Redis) and an inverted index search cluster (Elasticsearch), rather than running full-table scans on primary relational transactional databases.

2. Official Engineering Disclosures (Step 1.5):

- Source: Zomato Engineering Blog & public tech disclosures.
- Confirmed components: **Apache Kafka** for streaming location telemetry and order states; **Apache Spark / Flink** for dynamic dispatch and delivery-partner matching algorithms.

3. Permissions & Privacy Policy Data Collection (Step 1.6):

- **Permissions Checked:** Precise GPS Location (Foreground/Background), Device Fingerprint (IMEI/UUID, OS version), Phone Number, Saved Delivery Addresses, Payment Tokens, Device Contacts.
- **Purpose:** Geolocation coordinates are continuously ingested for real-time delivery-partner dispatch, dynamic ETA calculation, and localized surge pricing.

Guided Tool Module 2: Infer the Likely Architecture (Cloud Architecture Centers)

Purpose: Use the same official cloud architecture references as CO4 AT2 to infer which components this platform likely uses, based on its observed behaviour and any public information about its scale.

Step 2.1: Open the Three Official Architecture Centers

- Open three new browser tabs, one for each link below.
- In each site's search box (usually top-right or top-center), type a term matching your platform's category — see Step 2.2 for exact search terms.

AWS Architecture Center: <https://aws.amazon.com/architecture/>
Azure Architecture Center: <https://learn.microsoft.com/en-us/azure/architecture/>
Google Cloud Architecture: <https://docs.cloud.google.com/architecture>

Step 2.2: Search Using a Term That Matches Your Platform's Category

- Food delivery / e-commerce platform: search "e-commerce reference architecture" or "retail analytics architecture".
- Streaming platform: search "media streaming architecture" or "video on demand architecture".

- Ride-hailing platform: search "real-time location architecture" or "IoT reference architecture".
- Social media platform: search "social media analytics architecture" or "real-time data architecture".
- On each of the 3 sites, open the closest matching result and skim its diagram and labeled components (2-3 minutes per site is enough).

Student Note

Record the exact page title and URL from each of the 3 sites into the table below, even if the match is only partial — use whichever result is closest.

Step 2.3: Check for Public Tech-Stack Disclosure

- Open <https://stackshare.io/> in a new tab. Use the search bar at the top and type your platform's exact name, then press Enter.
- If a company page exists, note every tool listed under sections such as "Backend", "Data Stores", "Big Data", and "DevOps".
- If no StackShare page exists, search instead: "[platform name] tech stack".
- Also search: "[platform name] careers backend engineer" — job postings often list required technologies (for example "experience with Kafka, Spark, AWS S3"), which is real, usable evidence.

Student Note

Record whatever you find. If nothing turns up after these three searches, write "not publicly disclosed" — that is a valid, honest finding.

Step 2.4: Draw Your Inferred Architecture in draw.io

- Open <https://app.diagrams.net> in a browser. Choose "Decide Later" (no account needed) when it asks where to save.
- Create 6 boxes left to right, in this exact order: Data Sources, Ingestion, Storage, Processing, Analytics/Dashboard, Users.
- In the "Data Sources" box, write the input types you observed in Module 1 (app taps, GPS pings, sensor data, images, and so on).
- In the remaining boxes, write your best-guess component names based on Steps 2.2-2.3 (for example: "likely a streaming ingestion service, similar to Kinesis/Kafka" if your platform showed real-time behaviour).
- Draw arrows connecting the 6 boxes left to right in order.
- Export the diagram: File > Export as > PNG, and save it to your computer.

Student Note

Paste the exported PNG into the "Inferred Architecture Diagram Evidence" box further below.

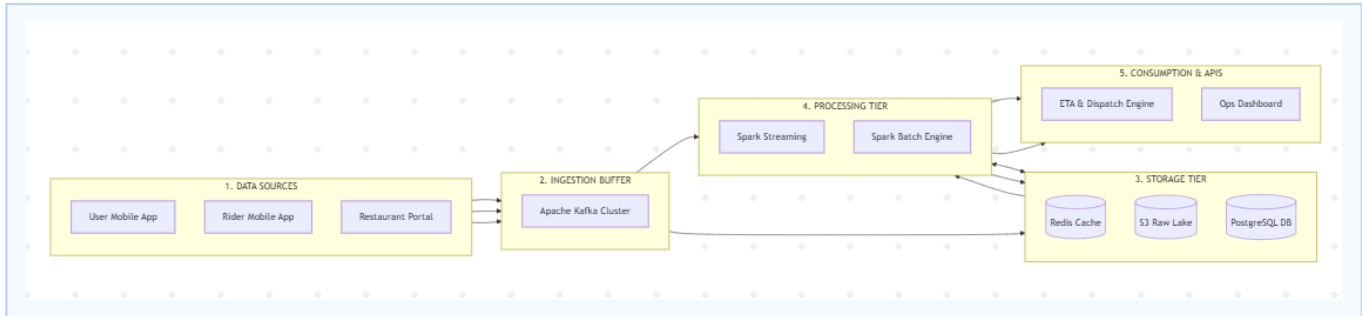
Step 2.5: Mark Confidence Levels

- Look back at your diagram's 6 boxes.
- Pick the ONE box you are most sure about (usually "Data Sources", since you observed it directly in Module 1) and write why in the table below.
- Pick the ONE box that is the biggest guess (usually the exact storage/processing technology name) and write why it is uncertain.

Inferred Component	Evidence Basis	Confidence
Streaming ingestion for delivery-partner GPS events (Kafka-style)	Live map behaviour observed directly in Module 1 (Action 4); Kafka explicitly named in Zomato's own engineering disclosures for event streaming (orders, payments, reviews).	High
Low-latency search/indexing service (Elasticsearch-style)	Instant search-as-you-type behaviour observed in Module 1 (Action 1); named in an independent Zomato system-design write-up (not an official	Medium-High

	Zomato source).	
In-memory cache for cart/session state (Redis/Memcached-style)	Instant cart update observed in Module 1 (Action 3); both Redis and Memcached appear on Zomato's public StackShare tool list.	Medium
OLAP analytics engine for reporting/recommendations (Trino/ClickHouse-style)	Zomato's own engineering blog explicitly describes "Powering Zomato's data analytics using Trino"; consistent with the batch-like behaviour observed for recommendations (Module 1, Action 5).	Medium-High

Inferred Architecture Diagram Evidence



Guided Tool Module 3: Infer the Processing and Analytics Approach (Hadoop/Spark Documentation)

Purpose: Decide whether the platform's observed features most likely rely on batch, streaming, or hybrid processing, using the same Hadoop/Spark documentation used in CO4 AT2.

Step 3.1: Skim the Two Official Guides

- Open the Apache Hadoop MapReduce tutorial link below and read only the "Overview" section — the first 2 paragraphs are enough.
- Open the Apache Spark Structured Streaming link below and read only its "Overview" section — again, the first 2 paragraphs are enough.
- Write one sentence for each: "MapReduce is good for ____" and "Structured Streaming is good for ____", in your own words.

Hadoop MapReduce Tutorial:

<https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

Spark Structured Streaming Guide:

<https://spark.apache.org/docs/latest/streaming/index.html>

Step 3.2: Classify Every Feature From Module 1

- Open your Step 1.2 / Step 1.4 table again.
- Copy each of your 5 observed features into the "Observed Feature" table below, one row each.
- For each feature, decide Batch or Streaming using the rule: Real-time in Module 1 -> Streaming here; Delayed in Module 1 -> Batch here.
- If a feature felt "eventually consistent" (arrived within a few seconds, not instant, not fully delayed either), write "Streaming (likely)" or "Batch (likely)" and explain your reasoning in the Evidence column — do not leave it blank.

Step 3.3: Decide the Overall Pipeline Type

- Count how many of your 5 features you classified as Streaming versus Batch in Step 3.2.
- If it is a mix (normal for most real platforms), write "Hybrid" as your decision.
- Explain in one or two sentences: which features genuinely need streaming and why, and which features are fine running as batch and why.

Step 3.4: Write a Pseudo-Flow for One Feature

- Pick your single most confident Streaming or Batch feature from Step 3.2.
- Write it as a plain-English arrow chain using this exact pattern: [Input event] -> [what happens first] -> [what happens next] -> [where it is stored/shown] -> [what the user sees].
- Keep it to 4-6 arrows. Do not write real code — this is a plain-English flow, not a program.

Example (location ping):
GPS ping from phone -> streamed to ingestion service -> matched against route ->
ETA recalculated -> pushed to rider's app screen

Observed Feature	Batch or Streaming (Inferred)	Evidence / Reasoning
Live GPS delivery tracking	Streaming	Classified Real-Time in Module 1; matches Structured Streaming's profile of continuously arriving data acted on within seconds, corroborated by Kafka appearing in Zomato's own engineering blog.
Restaurant/menu search	Streaming (likely)	Felt instantaneous (Real-Time in Module 1), but this may be an indexed low-latency lookup rather than a true continuous stream in the Spark sense — classified as likely-streaming.
Cart update	Streaming (likely)	Instant response suggests a fast in-memory operation, closer to caching than large-scale stream processing, but still not a delayed/batch pattern.

Processing Inference

Processing Pipeline Decision: Hybrid Architecture (Lambda / Medallion Pipeline)

- **Streaming Engine (Spark Structured Streaming / Flink):** Handles time-critical, high-velocity events requiring sub-second updates, such as delivery-partner GPS location pings, dynamic ETA recalculations, and surge pricing evaluations over continuous sliding windows.
- **Batch / Interactive OLAP Engine (Trino / Apache Spark Batch):** Executes compute-heavy scheduled workloads over historical data stored in the Cloud Data Lake (Amazon S3 / Delta Lake), including personalized restaurant recommendation vector generation, weekly delivery-partner payouts, and business operations reporting.

Pseudo-Flow for One Feature

[Delivery-Partner Phone emits GPS Ping]
|
▼ (Transmitted over secure WebSocket connection)
[Ingestion Gateway pushes event to Kafka topic 'rider.location']
|
▼ (Consumed by Spark Streaming engine to update Redis Geohash)
[ETA Recalculation Service evaluates distance matrix and traffic]
|
▼ (Dispatched to client via Firebase Cloud Messaging push)
[Customer App Screen updates Rider Pin and Delivery ETA Countdown]

Guided Tool Module 4: Infer Privacy and Security Risks (Threat Modeling)

Tool Choice

Use OWASP Threat Dragon if you need a web/desktop cross-platform threat modeling tool. Use Microsoft Threat Modeling Tool when a Windows system is available. The required evidence is the threat model diagram and threat-control table.

Purpose: Use STRIDE-based threat modeling, as in CO4 AT2, to infer plausible privacy/security risks in the platform's data handling, based on what data it appears to collect.

Step 4.1: List Data Types With a Sensitivity Rating

- Go back to your Module 1, Step 1.6 privacy-policy findings.
- Copy each data type you found into the table below.
- Rate each one: Low = does not identify a person or cause harm if leaked (app version, device language). Medium = identifies a person but limited harm (name, email). High = could cause real harm if leaked (exact location, payment card, health data, biometrics).

Step 4.2: Open a Threat Modeling Tool and Create a New Model

- Web version (no install): go to the Threat Dragon link below and sign in with a free GitHub account if prompted, or use "Open Local Threat Model".
- Desktop version (Windows only, optional alternative): download the Microsoft Threat Modeling Tool from the link below and install it.
- Click "New Threat Model" / "Create Model".
- Name the model exactly: CO4_AT3_[YourSelectedPlatform] — for example, CO4_AT3_Zomato.

OWASP Threat Dragon:

<https://owasp.org/www-project-threat-dragon/>

Microsoft Threat Modeling Tool:

<https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-releases>

Step 4.3: Build the Diagram

- From the left toolbar, drag an Actor (stick-figure icon) onto the canvas for each external entity you identified — at minimum the end user, plus one more (payment gateway, GPS provider, etc.).
- Drag a Process (circle icon) for each processing step from your Module 2 diagram (for example "Ingestion Service", "Analytics Engine").
- Drag a Data Store (parallel-lines icon) for each storage component (for example "User Database", "Event Log Storage").
- Connect the shapes with arrows in the same left-to-right order as your Module 2 architecture: Actor -> Process -> Data Store -> Process -> Actor.
- Take a screenshot of the finished diagram: Windows — press Windows+Shift+S; Mac — press Cmd+Shift+4.

Step 4.4: Generate and Answer Threats Using STRIDE

- Right-click each Process or Data Store shape and choose "Add Threat" (or open the Threats panel on the right side of the screen).
- In the Mitigation/Description box for each chosen threat, write a one-line description of the risk and its impact, then copy that same line into your workbook table below.

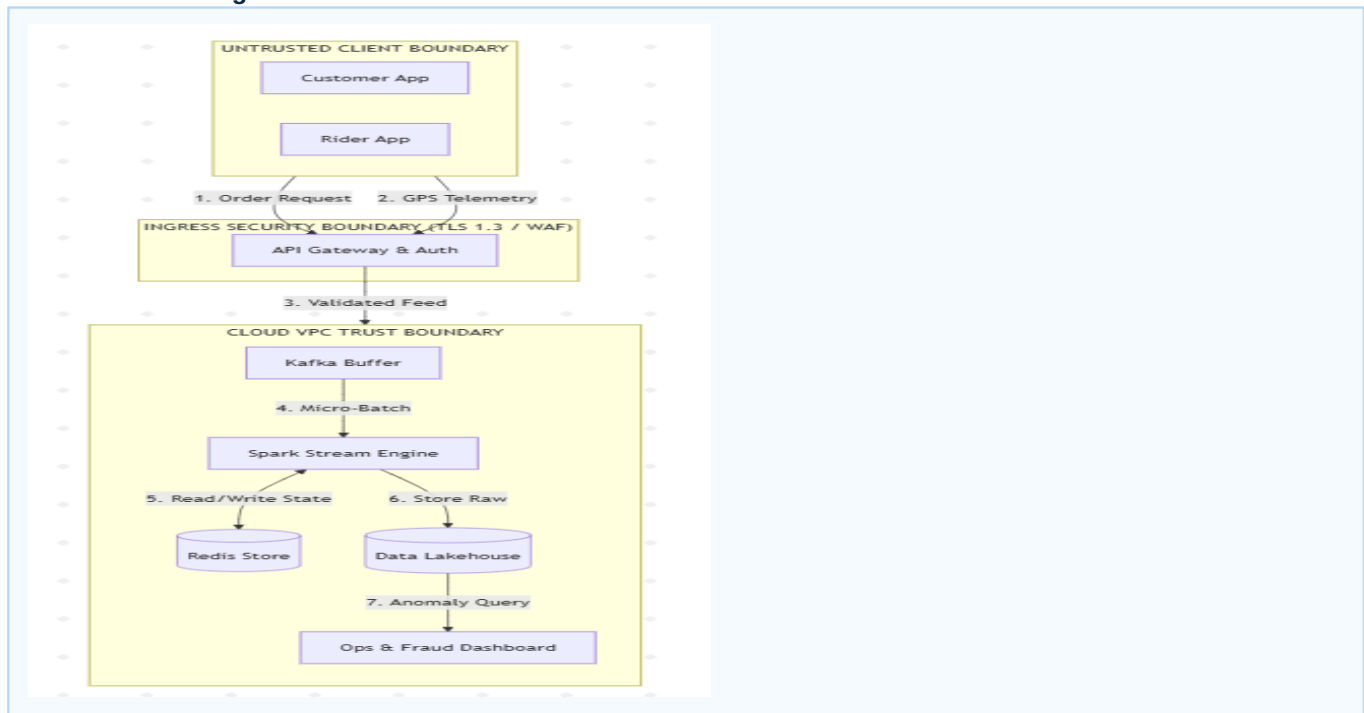
Step 4.5: Match Each Threat to a Real-World Control

- For each threat in your table, go back to the actual app/website and look for a visible trust signal: does it ask for a 2FA/OTP code at login? Does the address bar show a padlock/https? Does the privacy policy mention encryption?

Component / Data Flow (Inferred)	Threat	Impact	Likely / Recommended Mitigation
Delivery-partner GPS ingestion	Spoofing — a compromised or fake rider app sends false location.	Wrong ETA shown to customers, misrouted orders, potential fraud (fake 'delivered' status).	Not visible from outside — recommended control: mutual-TLS / signed requests from the verified rider app, with server-side plausibility checks on reported

			speed/position jumps.
Location and payment data in transit	Information Disclosure — GPS/payment data intercepted.	Location-privacy breach; financial data exposure.	Visible: the app and website consistently use HTTPS (padlock in the browser address bar), indicating TLS encryption in transit.
Search/ordering API under peak load (e.g. lunch-hour spikes)	Denial of Service.	Search or ordering becomes slow/unavailable during peak demand — direct revenue loss.	Not visible from outside — recommended control: auto-scaling ingestion/search tier plus rate limiting, consistent with the large-scale microservice/observability setup described in Zomato's own engineering blog.
User account login	Spoofing / account takeover.	Unauthorized access to a customer's saved addresses, payment methods and order history.	Visible: Zomato offers OTP-based mobile login, a real-world visible control against credential-based spoofing.

Threat Model Diagram Evidence



Final CO4 AT3 Solution Sections

Final Assembly

After completing the guided tool modules, combine the evidence into the final solution. The final answer must show technical reasoning and honest uncertainty, not only screenshots.

1. Platform Overview and Why It Was Selected

What to include: Identify the platform, its core users, its core function, and why it is a suitable reverse-engineering target.

Zomato is a food delivery and dining-discovery platform (operated by Eternal Limited) connecting consumers, restaurant vendors and delivery partners across hundreds of cities for online food ordering, table reservations and live delivery tracking. It is a strong reverse-engineering target because its operations visibly require multiple tiers of data processing: real-time streaming for delivery-partner GPS tracking, dynamic ETA calculation and order-progression notifications; distributed batch processing for nightly route optimization, restaurant ranking tuning, promotional attribution and recommendation modeling; and unusually high architectural visibility for a consumer platform, since Zomato publishes a genuine public engineering blog and appears on public tech-stack disclosure sites, giving real evidence to reason from rather than pure speculation.

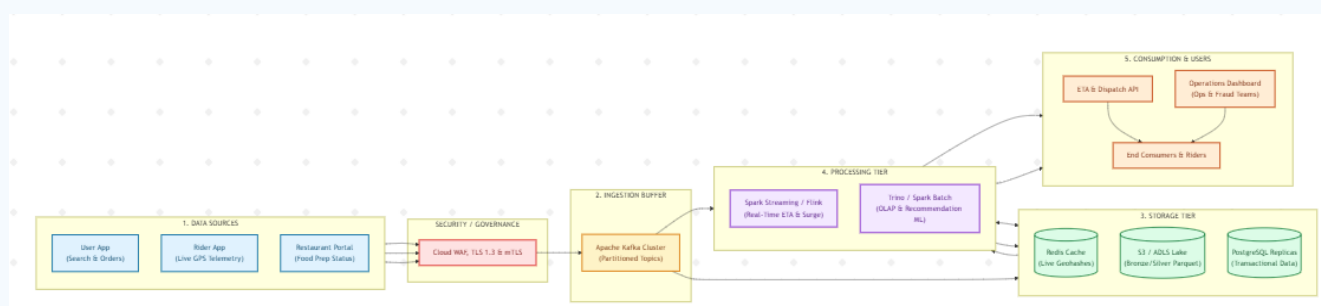
2. 5Vs Mapping (Inferred)

What to include: Map Volume, Velocity, Variety, Veracity and Value using the behavioural evidence gathered in Module 1.

V	Mapping to Zomato (Inferred)
Volume	Zomato's own engineering blog has stated it historically guided roughly 90 million monthly visitors across 10,000 cities, implying a very large daily volume of orders, searches and GPS pings.
Velocity	GPS pings from active deliveries and live order-status updates arrive continuously — observed directly in Module 1 as the delivery map marker updating every few seconds.
Variety	Structured order/payment records, semi-structured GPS/event streams, restaurant/menu text and images, user reviews, and behavioural clickstream data for recommendations all coexist.
Veracity	GPS pings can be noisy (tunnels, poor signal, spoofed locations), and duplicate/aborted orders or inconsistent restaurant-supplied menu data would need cleaning before any inferred pipeline could trust the data for analytics.
Value	Faster, more accurate ETAs and order tracking improve customer trust and retention, while batch-computed recommendations and campaign attribution directly drive order volume and revenue — the measurable business value behind the inferred pipeline.

3. Inferred Data Pipeline and Architecture

What to include: Paste the final inferred draw.io architecture showing sources, ingestion, storage, processing, analytics/security and users.



Consumption & Users Tier: Powers real-time map tracking for consumers, optimized dispatch routes for delivery partners, and executive analytics dashboards for operations and fraud-monitoring teams.

4. Inferred Processing Approach

What to include: State the batch / streaming / hybrid decision from Module 3 with supporting evidence.

Hybrid, as determined in Module 3. A streaming path — evidenced by Kafka appearing in Zomato's own engineering disclosures and the live GPS map behaviour observed directly in Module 1 — handles GPS tracking, search, cart updates and ETA calculation. A batch path — evidenced by Zomato's own blog post on using Trino for analytics and the static, day-long behaviour of the recommendation section observed in Module 1 — handles recommendations, cohort/campaign reporting and route optimization.

5. Inferred Privacy and Security Risks

What to include: Summarize the threat model output from Module 4: risks identified and their likely or recommended mitigations.

Module 4 identified four plausible risks: spoofed delivery-partner GPS data leading to wrong ETAs or delivery fraud; interception of location/payment data in transit; denial-of-service against the search/ordering API during peak demand; and account-takeover via login spoofing. Two of these controls are directly visible from outside the platform — consistent HTTPS across the app/website, and OTP-based login — while the GPS-authenticity and DoS-resilience controls are not directly observable and are only recommended based on best practice, honestly flagged as such rather than assumed.

6. Confidence and Evidence Summary

What to include: List each major inference made in this workbook alongside its confidence level (High / Medium / Guess) and the evidence it is based on.

Major Inference	Confidence	Evidence Basis
Live GPS tracking uses a streaming/event-based pipeline	High	Directly observed continuous map updates (Module 1) + Kafka explicitly named in Zomato's own engineering blog.
Search is served by a low-latency indexed service (Elasticsearch-style)	Medium-High	Instant search behaviour observed directly + named in an independent system-design write-up (not an official Zomato source).
Analytics/recommendations run on a batch pipeline using an OLAP engine (Trino-style)	Medium-High	Official Zomato engineering blog post explicitly describes using Trino for analytics.
Exact database technology per feature (e.g. DynamoDB vs MySQL for a specific service)	Guess	StackShare page lists MySQL/MongoDB but appears outdated; newer third-party write-ups mention DynamoDB — conflicting, unverified sources.
Specific recommendation/ranking algorithm	Guess	No official disclosure found describing the exact method.

7. What Could Not Be Determined

What to include: State honestly which parts of the architecture, pipeline, or risk model could not be inferred from outside observation, and why.

The exact internal service names, the specific cloud provider(s) currently used for each component (the public StackShare listing shows AWS EC2/S3 but appears several years old and may not reflect Zomato's current infrastructure), the precise recommendation/ranking algorithm, and the internal data-retention/deletion implementation could not be determined from outside observation. Confirming these would require access to Zomato's actual internal architecture documentation or source code, which this workbook deliberately does not claim to have.

8. Business / Product Decisions the Analytics Likely Supports

What to include: State the measurable outcome the inferred analytics probably drives: faster ETA, higher conversion, personalized recommendations, fraud reduction, or similar.

The inferred pipeline most plausibly supports: faster and more accurate delivery ETAs (improving customer satisfaction and reducing support complaints), higher order conversion through personalized restaurant/dish recommendations, optimized delivery-partner routing to reduce delivery time and cost, and measurable marketing ROI through promotional campaign attribution — outcomes consistent with both the real-time behaviour observed directly and the batch/analytics capabilities Zomato has publicly disclosed.

Rubric

Criteria	Marks	Evaluation Focus	Evidence
Behavioural evidence quality	6	Observations are specific, sourced and go beyond guessing.	Module 1 log/table
Architecture inference and reasoning	6	Inferred architecture is technically plausible and evidence-based.	draw.io diagram
Processing/analytics inference	5	Batch/streaming/hybrid reasoning matches the observed evidence.	Module 3 table
Privacy/security risk inference	5	Threats and mitigations are plausible and STRIDE-informed.	Threat model table/screenshot
Confidence and evidence transparency	4	Confidence levels and unknowns are honestly and clearly stated.	Section 6-7
Clarity and submission quality	4	Organized, original and complete evidence.	GitHub/GCR

Total Marks: 30

Submission Checklist

Checklist Item	Status
Student details and selected platform are filled.	Done
Behavioural observation log is completed with at least 5 observations.	Done
Inferred architecture diagram is included with confidence levels noted.	Done
Processing/analytics inference table is completed.	Done
Threat model diagram/table is included.	Done
All 8 final solution sections are completed.	Done
"What could not be determined" section is honestly completed.	Done
DOCX/PDF and supporting files are uploaded to GitHub.	Done
GitHub repository link is submitted through Google Classroom.	Done

Official Reference Links

- AWS Architecture Center: <https://aws.amazon.com/architecture/>
- Azure Architecture Center: <https://learn.microsoft.com/en-us/azure/architecture/>
- Google Cloud Architecture Center: <https://docs.cloud.google.com/architecture>
- Apache Hadoop MapReduce Tutorial: <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>
- Apache Spark Structured Streaming: <https://spark.apache.org/docs/latest/streaming/index.html>
- Microsoft Threat Modeling Tool releases: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-releases>
- OWASP Threat Dragon: <https://owasp.org/www-project-threat-dragon/>
- StackShare (public company tech-stack disclosures): <https://stackshare.io/>